INSTITUTO FEDERAL
Paraná
Campus Pinhais

INSTITUTO FEDERAL DO PARANÁ

BEATRIZ REGINA NAHAS
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

MANUAL 2 – EBPF PARA SEGURANÇA

PINHAIS
2026

BEATRIZ REGINA NAHAS
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

MANUAL 2 – EBPF PARA SEGURANÇA

Manual produzido no âmbito do projeto de extensão do minicurso Introdução ao eBPF, apresentado à disciplina de Práticas de Extensão V, do curso de Bacharelado em Ciência da Computação do Instituto Federal do Paraná, Campus Pinhais, sob a orientação do professor Guilherme Werneck de Oliveira.

Orientador: Prof. Guilherme Werneck de Oliveira

PINHAIS
2026

SUMÁRIO

1 INTRODUÇÃO **4**
1.1 EBPF NO CONTEXTO DE SEGURANÇA 4
1.2 CASOS DE USO DE SEGURANÇA 5
1.3 VANTAGENS DO EBPF PARA SEGURANÇA 6
2 PRÉ-REQUISITOS **7**
2.1 CONHECIMENTOS NECESSÁRIOS 7
2.2 AMBIENTE LINUX 7
3 FUNDAMENTOS DE SEGURANÇA COM EBPF **9**
3.1 MONITORAMENTO DE EVENTOS 9
3.2 RASTREAMENTO DE PROCESSOS 10
3.3 MONITORAMENTO DE CHAMADAS DE SISTEMA 10
3.4 CONTROLE DE ACESSO 11
3.5 DETECÇÃO DE COMPORTAMENTOS SUSPEITOS 12
4 EBPF E O KERNEL LINUX **14**
4.1 INTERAÇÃO COM O KERNEL 14
4.2 HOOKS 15
4.3 PROGRAMAS EBPF 16
4.4 VERIFIER 16
4.5 MAPS 17
5 MECANISMOS DE SEGURANÇA **19**
5.1 EBPF LSM 19
5.2 *CGROUP HOOKS* 20
5.3 SECCOMP E EBPF 20
5.4 CONTROLE DE PROCESSOS E ACESSO A RECURSOS 21
6 DETECÇÃO E MONITORAMENTO **23**
6.1 DETECÇÃO DE EXECUÇÃO DE PROCESSOS 23
6.2 MONITORAMENTO DE ARQUIVOS, CHAMADAS DE SISTEMA E CONEXÕES 24
6.3 DETECÇÃO DE COMPORTAMENTO ANÔMALO 25
6.4 AUDITORIA DE EVENTOS 26
7 FERRAMENTAS DE SEGURANÇA **28**
7.1 BCC 28
7.2 BPFTRACE 29
7.3 LIBBPF 30
7.4 FALCO 30
7.5 TETRAGON 31
8 EBPF EM CONTAINERS E KUBERNETES **34**
8.1 SEGURANÇA DE CONTÊINERES 34
8.2 SEGURANÇA EM KUBERNETES 35
8.3 POLÍTICAS DE SEGURANÇA 35
8.4 DETECÇÃO DE ATIVIDADES SUSPEITAS 36
9 EXEMPLOS PRÁTICOS **38**
9.1 MONITORAMENTO DE PROCESSOS E ARQUIVOS 38
9.2 RASTREAMENTO DE CHAMADAS DE SISTEMA 39
9.3 DETECÇÃO DE CONEXÕES SUSPEITAS 40
10 SEGURANÇA DO PRÓPRIO EBPF **42**

10.1 PRIVILÉGIOS NECESSÁRIOS 42
10.2 EBPF VERIFIER 43
10.3 RISCOS DE SEGURANÇA 44
10.4 LIMITAÇÕES 45
11 CONCLUSÃO **47**

1 INTRODUÇÃO

O *extended Berkeley Packet Filter* (eBPF) é uma tecnologia do kernel Linux que permite a execução de programas verificados e seguros diretamente no espaço do kernel, sem a necessidade de modificá-lo ou carregar módulos adicionais (Linux Kernel Documentation, 2024). Originada do BSD Packet Filter (BPF) proposto por McCanne e Jacobson (1993) para captura eficiente de pacotes em nível de usuário, a tecnologia evoluiu amplamente e passou a suportar casos de uso muito além da análise de rede: rastreamento de desempenho, observabilidade e, de forma crescente, segurança de sistemas (GREGG, 2019).

Neste contexto, o presente relatório documenta o minicurso introdutório de eBPF realizado na *Semana de Ciência e Tecnologia* (SciTec) do Instituto Federal do Paraná, câmpus Pinhais, em 2026. O minicurso teve como base o Trabalho de Conclusão de Curso de Alves e Plinta (2026), cujo foco é o monitoramento de funções virtualizadas de redes (NFV), e foi expandido para cobrir aplicações voltadas à área de segurança da informação. O material prático e o repositório de código utilizados estão disponíveis em Plinta e Nahas (2026).

1.1 EBPF NO CONTEXTO DE SEGURANÇA

A segurança de sistemas operacionais depende, em grande medida, da capacidade de observar e controlar eventos internos ao kernel com baixa latência e mínima interferência sobre as cargas de trabalho em produção (BEYER *et al.*, 2016). Abordagens tradicionais — como módulos de kernel dedicados, agentes em espaço de usuário ou chamadas de sistema adicionais — impõem sobrecarga considerável ou exigem privilégios que ampliam a superfície de ataque (CASSAGNES *et al.*, 2020).

O eBPF resolve essa tensão ao fornecer um ambiente de execução sandboxado dentro do próprio kernel (Linux Kernel Documentation, 2024). Antes de ser executado, cada programa eBPF passa por um verificador (*verifier*) estático que garante terminação, ausência de acessos a memória inválidos e respeito às políticas de segurança do sistema — tornando a injeção de código malicioso por essa via estruturalmente inviável (GREGG, 2019). Além disso, o mecanismo BPF CO-RE (*Compile Once – Run Everywhere*) permite que programas compilados uma única vez sejam executados em diferentes versões do kernel sem recompilação, o que facilita a implantação uniforme de políticas de segurança em ambientes heterogêneos (NAKRYIKO, 2020).

Ferramentas consolidadas como BCC (IO Visor Project, s.d.) e bpftrace (bpftrace Contributors, s.d.), construídas sobre a biblioteca libbpf (libbpf Contributors, s.d.), tor-

naram o desenvolvimento de programas eBPF acessível a engenheiros e pesquisadores de segurança, acelerando a adoção da tecnologia nesse domínio.

1.2 CASOS DE USO DE SEGURANÇA

O eBPF encontra aplicação em diversas categorias de segurança de sistemas e redes.

Filtragem e controle de pacotes de rede. Por meio do *eXpress Data Path* (XDP), programas eBPF são anexados à interface de rede antes mesmo que os pacotes sejam entregues à pilha de rede do kernel, possibilitando bloqueio de tráfego malicioso — como ataques de negação de serviço (DDoS) — com latência mínima (Linux Kernel Documentation, 2024). Essa abordagem supera em desempenho soluções baseadas em *iptables* ou *nftables* para cenários de alta taxa de pacotes (GREGG, 2020).

Monitoramento de chamadas de sistema (*syscall*). Programas eBPF podem ser vinculados a pontos de rastreamento (*tracepoints*) e sondas (*kprobes/uprobes*) sobre chamadas de sistema, permitindo detectar comportamentos anômalos em tempo real — como tentativas de escalonamento de privilégios, acesso a arquivos sensíveis ou execução de binários não autorizados (GREGG, 2019). Essa capacidade é explorada por soluções de segurança em tempo de execução (*runtime security*), como Falco e Cilium, que baseiam suas regras de detecção em programas eBPF.

Aplicação de políticas de rede em ambientes de contêineres. Em infraestruturas Kubernetes, o eBPF substitui com vantagens o modelo tradicional de *kube-proxy* baseado em *iptables*, implementando políticas de rede (*NetworkPolicies*) com menor sobrecarga e maior expressividade (CASSAGNES *et al.*, 2020). A visibilidade granular sobre fluxos de comunicação entre pods viabiliza também a detecção de movimentação lateral em ataques internos ao cluster.

Auditoria e rastreamento forense. A capacidade de registrar, em mapas eBPF, eventos de sistema com alta fidelidade e sem impacto perceptível sobre as aplicações torna a tecnologia adequada para coleta de evidências forenses em ambientes de produção (MAJORS; FONG-JONES; MIRANDA, 2022). Diferentemente de agentes em espaço de usuário, que podem ser suprimidos por um invasor com privilégios suficientes, programas eBPF no kernel oferecem maior resistência a tentativas de evasão.

1.3 VANTAGENS DO EBPF PARA SEGURANÇA

Em comparação com abordagens convencionais, o uso de eBPF em segurança apresenta um conjunto de vantagens que justificam sua adoção crescente.

Baixo impacto sobre o desempenho. A execução de programas eBPF ocorre no kernel, evitando as trocas de contexto (*context switches*) necessárias em soluções baseadas em espaço de usuário. Cassagnes *et al.* (2020) demonstraram que o eBPF impõe sobrecarga significativamente menor do que ferramentas tradicionais de monitoramento, tornando-o adequado para ambientes de produção com requisitos rígidos de latência.

Segurança por construção. O verificador do kernel rejeita programas que possam comprometer a estabilidade ou a segurança do sistema, garantindo que apenas código validado seja executado (Linux Kernel Documentation, 2024). Essa propriedade distingue o eBPF de módulos de kernel convencionais, que operam sem tais restrições e representam risco elevado quando mal implementados.

Portabilidade e manutenibilidade. O BPF CO-RE (NAKRYIKO, 2020) elimina a necessidade de recompilar programas para cada versão de kernel, reduzindo o custo operacional de manutenção de ferramentas de segurança distribuídas em frotas heterogêneas de servidores.

Programabilidade e extensibilidade. Ao contrário de soluções estáticas, políticas de segurança baseadas em eBPF podem ser atualizadas dinamicamente — sem reinicialização do sistema ou interrupção dos serviços — respondendo a novas ameaças com agilidade (GREGG, 2019). Ecossistemas como o BCC (IO Visor Project, s.d.) e o bpftrace (bpftrace Contributors, s.d.) fornecem abstrações de alto nível que reduzem a curva de aprendizado para equipes de segurança sem experiência prévia em programação de kernel.

Visibilidade unificada. Por operar no ponto central de controle do sistema operacional, um único programa eBPF pode correlacionar eventos de rede, sistema de arquivos, agendador de processos e chamadas de sistema, oferecendo uma visão holística do comportamento do sistema que ferramentas compartimentadas dificilmente alcançam (MAJORS; FONG-JONES; MIRANDA, 2022).

2 PRÉ-REQUISITOS

Este capítulo descreve os conhecimentos prévios recomendados e o ambiente necessário para acompanhar o minicurso e reproduzir os exemplos práticos apresentados.

2.1 CONHECIMENTOS NECESSÁRIOS

O minicurso foi concebido para participantes com familiaridade básica com sistemas operacionais Linux e programação em linguagem C. Não é exigida experiência prévia com desenvolvimento de kernel ou com eBPF. Os tópicos a seguir representam a base mínima esperada.

Linha de comando Linux. O participante deve ser capaz de navegar pelo sistema de arquivos, executar comandos com privilégio de superusuário (`sudo`), instalar pacotes com o gerenciador de pacotes da distribuição e redirecionar a saída de programas (Linux man-pages project, 2024).

Linguagem C — nível introdutório. Programas eBPF são escritos em um subconjunto restrito de C e compilados para *bytecode* BPF (Linux Kernel Documentation, 2024). É suficiente compreender tipos primitivos, ponteiros, estruturas (`struct`) e a diferença entre espaço de usuário e espaço de kernel. Conceitos avançados, como alocação dinâmica de memória ou programação multithread, não são necessários neste nível introdutório.

Conceitos básicos de sistemas operacionais. É recomendável ter noção do que é uma chamada de sistema (*syscall*), como processos são escalonados e qual o papel do kernel na mediação entre aplicações e o hardware (GREGG, 2020). Esses conceitos contextualizam os pontos de ancoragem (*hooks*) nos quais programas eBPF são inseridos.

Noções de redes. Para os exemplos voltados à área de segurança de rede, é útil compreender o modelo de camadas TCP/IP, o que é um pacote e como endereços IP e portas identificam conexões (GREGG, 2019). Esse conhecimento não é estritamente obrigatório para os primeiros exercícios, mas facilita a interpretação dos resultados obtidos.

2.2 AMBIENTE LINUX

Todos os exemplos foram desenvolvidos e testados em ambiente Linux com kernel versão 5.15 ou superior, que é o mínimo recomendado para suporte estável às funcionalidades de eBPF utilizadas no minicurso (Linux Kernel Documentation, 2024). O código-fonte e os scripts de configuração do ambiente estão disponíveis no repositório público do minicurso (PLINTA; NAHAS, 2026).

Distribuição recomendada. Recomenda-se Ubuntu 22.04 LTS ou Debian 12, por oferecerem pacotes atualizados de ferramentas eBPF nos repositórios oficiais e ampla documentação de suporte. Outras distribuições baseadas em Debian ou em Red Hat (Fedora, RHEL 9+) são igualmente compatíveis, bastando adaptar os comandos de instalação de pacotes.

Dependências de software. Os seguintes pacotes devem estar instalados antes do início das atividades práticas:

- `clang` e `llvm` — compilador utilizado para gerar *bytecode* BPF a partir de código C (Linux Kernel Documentation, 2024);
- `libbpf-dev` — biblioteca de suporte a programas eBPF em espaço de usuário (libbpf Contributors, s.d.);
- `bpftrace` — linguagem de rastreamento de alto nível para eBPF (bpftrace Contributors, s.d.);
- `linux-headers` — cabeçalhos do kernel necessários para compilação de programas eBPF;
- `make` e `gcc` — utilitários de construção (*build*).

Em distribuições baseadas em Debian, a instalação pode ser realizada com:

```
sudo apt update
sudo apt install clang llvm libbpf-dev bpftrace \
                 linux-headers-$(uname -r) make gcc
```

Verificação de suporte do kernel. Antes de executar os exemplos, é possível verificar se o sistema expõe o sistema de arquivos virtual `/proc` e os pontos de rastreamento necessários com o comando:

```
ls /sys/kernel/debug/tracing/events
```

Caso o diretório não exista, o sistema de arquivos de depuração deve ser montado:

```
sudo mount -t debugfs debugfs /sys/kernel/debug
```

Uso em máquinas virtuais e contêineres. O eBPF pode ser utilizado em máquinas virtuais, desde que o kernel do sistema convidado tenha sido compilado com as opções `CONFIG_BPF`, `CONFIG_BPF_SYSCALL` e `CONFIG_BPF_JIT` habilitadas (Linux Kernel Documentation, 2024). Em contêineres Docker, é necessário que o contêiner seja iniciado com a capacidade (*capability*) `CAP_BPF` ou, alternativamente, em modo privilegiado (`-privileged`), pois operações eBPF requerem privilégios elevados (GREGG, 2019). Para fins didáticos, recomenda-se o uso de uma máquina virtual ou de uma instalação nativa, que oferecem maior visibilidade sobre o sistema e evitam restrições impostas pelas camadas de isolamento de contêineres.

3 FUNDAMENTOS DE SEGURANÇA COM EBPF

Este capítulo apresenta os mecanismos centrais pelos quais o eBPF viabiliza aplicações de segurança em sistemas Linux. Os conceitos são introduzidos de forma progressiva, partindo da observação passiva de eventos até técnicas de controle ativo e detecção de anomalias.

3.1 MONITORAMENTO DE EVENTOS

O modelo de execução do eBPF é orientado a eventos: um programa eBPF é sempre associado a um ponto de ancoragem (*hook*) no kernel e executado cada vez que o evento correspondente ocorre (Linux Kernel Documentation, 2024). Essa arquitetura torna o monitoramento de eventos uma operação de primeira classe, sem necessidade de *polling* periódico ou instrumentação invasiva das aplicações (CASSAGNES *et al.*, 2020).

Os principais tipos de pontos de ancoragem relevantes para segurança são:

- ***Tracepoints*** — pontos estáticos definidos pelo próprio kernel em locais estratégicos, como a entrada e saída de chamadas de sistema, eventos de escalonamento e operações de rede. Por serem estáveis entre versões do kernel, são preferidos para uso em produção (GREGG, 2019);
- ***Kprobes* e *kretprobes*** — sondas dinâmicas que podem ser inseridas no início ou no retorno de qualquer função do kernel, oferecendo visibilidade granular mesmo em pontos sem *tracepoint* estático (GREGG, 2020);
- ***Uprobes*** — equivalente das *kprobes* para funções em espaço de usuário, permitindo rastrear bibliotecas e aplicações sem modificá-las (bpftrace Contributors, s.d.);
- ***Perf events*** — eventos de hardware e software expostos pela infraestrutura `perf` do kernel, úteis para correlacionar comportamentos de segurança com métricas de desempenho (GREGG, 2019).

Dados coletados em cada evento são armazenados em estruturas chamadas **mapas BPF** (*BPF maps*), que funcionam como tabelas de chave-valor compartilhadas entre o programa eBPF no kernel e a aplicação em espaço de usuário responsável por exibir ou encaminhar os resultados (Linux Kernel Documentation, 2024). Essa separação entre coleta (kernel) e apresentação (espaço de usuário) é um dos pilares do modelo de segurança do eBPF.

3.2 RASTREAMENTO DE PROCESSOS

O rastreamento de processos é uma das aplicações mais diretas do eBPF em segurança e consiste em registrar o ciclo de vida de processos — criação, execução e encerramento — bem como suas relações de parentesco e os recursos que acessam (GREGG, 2019).

Ao se conectar aos *tracepoints* `sched_process_exec` e `sched_process_exit`, um programa eBPF obtém, para cada processo iniciado ou encerrado, informações como PID, UID, nome do executável e argumentos da linha de comando, sem qualquer modificação nas aplicações monitoradas (Linux Kernel Documentation, 2024). Essa visibilidade é especialmente valiosa em ambientes de contêineres, onde processos efêmeros dificultam o uso de ferramentas de auditoria tradicionais.

O trecho a seguir ilustra, em pseudocódigo BPF, como capturar a execução de um novo processo:

Listing 3.1 – Captura de evento de execução de processo via tracepoint.

```
SEC("tracepoint/syscalls/sys_enter_execve")
int trace_execve(struct trace_event_raw_sys_enter *ctx)
{
    struct event_t ev = {};
    ev.pid  = bpf_get_current_pid_tgid() >> 32;
    ev.uid  = bpf_get_current_uid_gid() & 0xFFFFFFFF;
    bpf_get_current_comm(&ev.comm, sizeof(ev.comm));
    bpf_perf_event_output(ctx, &events, BPF_F_CURRENT_CPU,
                          &ev, sizeof(ev));
    return 0;
}
```

Ferramentas como `execsnoop` e `opensnoop`, disponíveis no BCC (IO Visor Project, s.d.), implementam esse padrão e são amplamente utilizadas em investigações forenses e resposta a incidentes para reconstruir a sequência de execuções ocorridas em um sistema comprometido (MAJORS; FONG-JONES; MIRANDA, 2022).

3.3 MONITORAMENTO DE CHAMADAS DE SISTEMA

As chamadas de sistema (*syscalls*) constituem a única interface legítima entre aplicações em espaço de usuário e o kernel (GREGG, 2020). Monitorá-las é, portanto, equivalente a monitorar tudo que uma aplicação pode fazer no sistema — tornando os *tracepoints* de *syscall* o ponto de observação mais abrangente disponível para segurança em tempo de execução (GREGG, 2019).

Chamadas de particular interesse para segurança incluem:

- `execve` / `execveat` — execução de novos programas;
- `open` / `openat` — abertura de arquivos, relevante para detectar leitura de credenciais ou arquivos de configuração sensíveis;
- `connect` / `bind` — estabelecimento e escuta de conexões de rede, útil para identificar *backdoors* ou exfiltração de dados;
- `ptrace` — frequentemente abusada por *malware* para injeção de código em outros processos;
- `setuid` / `setgid` — alteração de identidade de processo, indicativa de tentativas de escalonamento de privilégios.

O bpftrace (bpftrace Contributors, s.d.) permite escrever scripts concisos para monitorar essas chamadas de sistema de forma interativa. O exemplo abaixo registra toda abertura de arquivo realizada por processos não privilegiados:

Listing 3.2 – Script bpftrace para monitorar abertura de arquivos por usuários não-root.

```
tracepoint:syscalls:sys_enter_openat
/uid != 0/
{
    printf("%s (pid=%d uid=%d): %s\n",
           comm, pid, uid, str(args->filename));
}
```

Em soluções de segurança em tempo de execução, como o Falco, regras expressas em alto nível são traduzidas internamente para programas eBPF que monitoram exatamente esse conjunto de chamadas de sistema, gerando alertas quando padrões suspeitos são detectados (CASSAGNES *et al.*, 2020).

3.4 CONTROLE DE ACESSO

Além de observar, o eBPF pode ser usado para *impor* políticas de acesso, indo além do monitoramento passivo. Essa capacidade é viabilizada pelo subsistema LSM BPF (*Linux Security Module* via eBPF), disponível a partir do kernel 5.7, que permite a criação de módulos de segurança programáveis sem necessidade de recompilar o kernel (Linux Kernel Documentation, 2024).

Com LSM BPF, um programa eBPF é conectado a *hooks* de segurança definidos pela infraestrutura LSM do kernel — os mesmos utilizados por SELinux e AppArmor —

e pode retornar um código de erro para **bloquear** a operação antes que ela seja concluída (GREGG, 2019). Exemplos de controles implementáveis por essa via incluem:

- negar a abertura de arquivos com base em atributos do processo solicitante (UID, GID, contexto de cgroup ou namespace);
- impedir a criação de sockets de rede por processos não autorizados;
- bloquear a execução de binários não listados em uma *allowlist* criptograficamente verificada;
- limitar o uso de chamadas de sistema perigosas por contêineres específicos, complementando perfis seccomp existentes.

No nível de rede, o XDP (*eXpress Data Path*) oferece controle de acesso em camada 2/3 com desempenho próximo ao do hardware, permitindo descartar pacotes maliciosos — como os de ataques de amplificação UDP — antes que consumam recursos da pilha de rede (Linux Kernel Documentation, 2024). Essa abordagem é utilizada em produção por empresas como a Cloudflare para mitigação de ataques DDoS em larga escala (Cloudflare, s.d.).

3.5 DETECÇÃO DE COMPORTAMENTOS SUSPEITOS

A combinação de monitoramento de eventos, rastreamento de processos e visibilidade sobre chamadas de sistema cria a base para sistemas de detecção de comportamentos suspeitos baseados em eBPF (MAJORS; FONG-JONES; MIRANDA, 2022). A premissa é que ataques e comprometimentos, por mais sofisticados que sejam, deixam rastros observáveis no nível do kernel que divergem do comportamento esperado de aplicações legítimas (BEYER *et al.*, 2016).

Padrões comuns de detecção incluem:

Execução de shells a partir de processos de serviço. Um servidor web que invoca `/bin/bash` é um forte indicador de exploração bem-sucedida de uma vulnerabilidade de execução remota de código. Um programa eBPF pode rastrear a árvore de processos em tempo real e alertar sempre que um processo filho inesperado for criado por um serviço monitorado (GREGG, 2019).

Acesso a arquivos sensíveis fora do padrão esperado. Leituras de `/etc/shadow`, `/proc/$PID/mem` ou arquivos de chaves privadas por processos que normalmente não os acessam indicam possível extração de credenciais (Linux man-pages project, 2024).

Estabelecimento de conexões de rede não previstas. Processos que normalmente não fazem conexões de saída e subitamente tentam contatar endereços externos podem estar executando *beaconing* de *Command and Control* (C2) (CASSAGNES *et al.*, 2020).

Modificação de binários do sistema em tempo de execução. Abertura em modo de escrita de executáveis localizados em `/usr/bin`, `/sbin` ou em diretórios de bibliotecas é característica de *rootkits* que tentam persistir no sistema (GREGG, 2020).

A eficácia da detecção aumenta quando múltiplos eventos são correlacionados em mapas BPF ao longo do tempo, permitindo identificar sequências de ações que, individualmente, seriam inofensivas, mas em conjunto configuram um padrão de ataque (MAJORS; FONG-JONES; MIRANDA, 2022). Essa correlação em espaço de kernel, com latência de microssegundos, é uma das vantagens do eBPF sobre soluções baseadas em coleta e análise assíncrona de logs (CASSAGNES *et al.*, 2020).

4 EBPF E O KERNEL LINUX

O eBPF transformou a maneira como programas interagem com o kernel Linux ao introduzir um modelo de extensão seguro, dinâmico e sem necessidade de modificar ou recompilar o kernel (Linux Kernel Documentation, 2024). Este capítulo descreve os mecanismos internos que tornam isso possível: a forma como programas eBPF são carregados e executados, os pontos em que se ancoram ao kernel, o processo de verificação de segurança e as estruturas de dados compartilhadas entre kernel e espaço de usuário.

4.1 INTERAÇÃO COM O KERNEL

Tradicionalmente, estender o comportamento do kernel Linux exigia a escrita de módulos (*kernel modules*), que operam com privilégios plenos e sem qualquer verificação prévia de segurança — uma falha em um módulo pode travar todo o sistema (GREGG, 2020). O eBPF oferece uma alternativa radicalmente diferente: programas são carregados em tempo de execução via a chamada de sistema `bpf(2)`, submetidos a verificação estática e, só então, executados no espaço do kernel em resposta a eventos específicos (Linux Kernel Documentation, 2024).

O fluxo de interação segue, de forma geral, as seguintes etapas:

1. O desenvolvedor escreve o programa eBPF em C restrito e o compila com `clang/llvm` para *bytecode* BPF (NAKRYIKO, 2020);
2. A aplicação em espaço de usuário carrega o *bytecode* no kernel via `bpf(2)`, passando-o pelo verificador;
3. O compilador JIT (*Just-In-Time*) do kernel traduz o *bytecode* para instruções nativas da arquitetura do processador, eliminando a sobrecarga de interpretação (Linux Kernel Documentation, 2024);
4. O programa é anexado a um *hook* e passa a ser executado cada vez que o evento correspondente ocorre;
5. Resultados são comunicados ao espaço de usuário por meio de mapas BPF ou do subsistema `perf` (GREGG, 2019).

A biblioteca libbpf (libbpf Contributors, s.d.) abstrai grande parte dessa interação, oferecendo uma API em C que cuida do carregamento, verificação e anexação de programas, além de implementar o BPF CO-RE (*Compile Once – Run Everywhere*), que

garante portabilidade do mesmo binário entre diferentes versões do kernel (NAKRYIKO, 2020).

4.2 HOOKS

Um *hook* é o ponto do kernel ao qual um programa eBPF é anexado e a partir do qual é disparado (Linux Kernel Documentation, 2024). A escolha do *hook* determina quais informações estarão disponíveis para o programa e quais ações ele poderá executar. Os principais tipos são descritos a seguir.

Tracepoints. Pontos de rastreamento estáticos inseridos pelos próprios mantenedores do kernel em locais estratégicos — entrada e saída de chamadas de sistema, eventos de escalonamento, operações de bloco e rede, entre outros. Por serem parte da ABI estável do kernel, não são removidos ou renomeados entre versões, tornando-os a opção mais robusta para uso em produção (GREGG, 2019).

Kprobes* e *kretprobes. Sondas dinâmicas que podem ser inseridas no início (*kprobe*) ou no retorno (*kretprobe*) de qualquer função do kernel, inclusive aquelas sem *tracepoint* estático (GREGG, 2020). Oferecem máxima cobertura, mas dependem de nomes de funções que podem mudar entre versões do kernel.

Uprobes* e *uretprobes. Equivalente das *kprobes* para funções em espaço de usuário. Permitem rastrear chamadas em bibliotecas compartilhadas (como `libc` ou `OpenSSL`) e aplicações sem modificá-las nem recompilá-las (bpftrace Contributors, s.d.).

XDP (*eXpress Data Path*). *Hook* de rede executado no driver da interface de rede, antes que o pacote seja entregue à pilha de rede do kernel. É o ponto de intervenção mais cedo e de menor latência disponível para processamento de pacotes, utilizado em filtragem, balanceamento de carga e mitigação de DDoS (Linux Kernel Documentation, 2024).

TC (*Traffic Control*). *Hook* na camada de controle de tráfego do kernel, após o processamento de camada 2 mas antes da entrega à aplicação. Oferece mais contexto sobre o pacote do que XDP, ao custo de execução um pouco mais tardia na pilha (GREGG, 2019).

LSM (*Linux Security Module*). *Hooks* de segurança utilizados por mecanismos como SELinux e AppArmor, disponíveis para eBPF a partir do kernel 5.7. Permitem criar políticas de controle de acesso programáveis que podem bloquear operações antes de sua conclusão (Linux Kernel Documentation, 2024).

4.3 PROGRAMAS EBPF

Um programa eBPF é escrito em um subconjunto restrito de C — sem chamadas de função arbitrárias, sem recursão, sem laços com número de iterações indeterminado — e compilado para um conjunto de instruções de 64 bits chamado *BPF bytecode* (Linux Kernel Documentation, 2024). Cada instrução opera sobre onze registradores de 64 bits (`r0`–`r10`), sendo `r10` um ponteiro somente leitura para o topo da pilha do programa e `r0` o registrador de valor de retorno (GREGG, 2019).

Os programas são organizados em tipos, e o tipo determina o *hook* compatível, o contexto passado como argumento e as operações permitidas. A Tabela 1 apresenta os principais tipos e suas aplicações.

Tabela 1 – Principais tipos de programas eBPF e suas aplicações.

Tipo	**Hook**	**Aplicação típica**
`BPF_PROG_TYPE_TRACEPOINT`	Tracepoints	Rastreamento de eventos
`BPF_PROG_TYPE_KPROBE`	Kprobes/uprobes	Sondagem de funções
`BPF_PROG_TYPE_XDP`	Driver de rede	Filtragem de pacotes
`BPF_PROG_TYPE_SCHED_CLS`	TC	Controle de tráfego
`BPF_PROG_TYPE_LSM`	Hooks LSM	Controle de acesso
`BPF_PROG_TYPE_PERF_EVENT`	Perf events	Profiling e monitoramento

Fonte: Adaptado de Linux Kernel Documentation (2024).

Programas eBPF podem chamar um conjunto curado de funções auxiliares do kernel — as *BPF helper functions* — para operações como obter o PID do processo atual (`bpf_get_current_pid_tgid`), ler memória do espaço de usuário de forma segura (`bpf_probe_read_user`) ou emitir dados para o espaço de usuário (`bpf_perf_event_output`) (Linux Kernel Documentation, 2024). O conjunto de *helpers* disponíveis varia conforme o tipo do programa, reforçando o princípio do menor privilégio.

4.4 VERIFIER

O verificador (*verifier*) é o componente do kernel responsável por analisar estaticamente todo programa eBPF antes de sua execução e rejeitar aqueles que possam comprometer a segurança ou a estabilidade do sistema (Linux Kernel Documentation, 2024). Essa análise ocorre em tempo de carregamento e é, portanto, transparente para as aplicações que consomem os dados coletados pelo programa (GREGG, 2019).

O verificador realiza duas passagens principais sobre o grafo de fluxo de controle do programa:

Primeira passagem — análise estrutural. Verifica que o grafo de fluxo de controle é um DAG (*Directed Acyclic Graph*), ou seja, que não existem ciclos não limitados que impeçam a terminação do programa. A partir do kernel 5.3, laços com número de iterações limitado e verificável são permitidos, mas o verificador precisa ser capaz de provar o limite superior (Linux Kernel Documentation, 2024).

Segunda passagem — análise de estados. Simula a execução do programa por todos os caminhos possíveis, rastreando o tipo e os possíveis valores de cada registrador e posição de pilha. São verificados, entre outros:

- ausência de acesso a ponteiros não inicializados ou fora dos limites permitidos (*out-of-bounds*);
- uso correto dos argumentos das *helper functions*;
- que o valor de retorno em `r0` é compatível com o tipo do programa;
- que ponteiros do kernel não são vazados para o espaço de usuário (Linux Kernel Documentation, 2024).

Se alguma dessas verificações falhar, o carregamento é rejeitado com uma mensagem de erro descritiva, que pode ser consultada por ferramentas como `bpftool` (IO Visor Project, s.d.). Essa propriedade garante que um programa eBPF com *bug* não derruba o kernel — o pior cenário é a rejeição na fase de carregamento (GREGG, 2019).

4.5 MAPS

Mapas BPF (*BPF maps*) são estruturas de dados genéricas de chave-valor alocadas no kernel e acessíveis tanto por programas eBPF quanto por aplicações em espaço de usuário (Linux Kernel Documentation, 2024). Eles são o único mecanismo pelo qual um programa eBPF pode manter estado entre execuções e comunicar resultados para fora do kernel (GREGG, 2019).

A criação de um mapa é feita via `bpf(BPF_MAP_CREATE, ...)`, especificando o tipo, os tamanhos de chave e valor e a capacidade máxima. O acesso em espaço de usuário ocorre por meio de descritores de arquivo, que podem ser fixados no sistema de arquivos virtual `/sys/fs/bpf` para persistir além do ciclo de vida do processo criador (Linux Kernel Documentation, 2024).

Os tipos de mapas mais utilizados em aplicações de segurança e observabilidade são:

- **`BPF_MAP_TYPE_HASH`** — tabela de dispersão de propósito geral; usada para rastrear estados de conexões, contagens por processo ou listas de bloqueio (Linux Kernel Documentation, 2024);
- **`BPF_MAP_TYPE_ARRAY`** — vetor indexado por inteiro; adequado para contadores globais e tabelas de configuração de tamanho fixo (GREGG, 2019);
- **`BPF_MAP_TYPE_PERF_EVENT_ARRAY`** — canal de comunicação assíncrona de alto desempenho para envio de eventos ao espaço de usuário; amplamente utilizado para notificação em tempo real de eventos de segurança (Linux Kernel Documentation, 2024);
- **`BPF_MAP_TYPE_RINGBUF`** — substituto moderno do `PERF_EVENT_ARRAY`, introduzido no kernel 5.8, com menor overhead de memória e garantia de ordenação de eventos (Linux Kernel Documentation, 2024);
- **`BPF_MAP_TYPE_LRU_HASH`** — variante com política de evicção LRU (*Least Recently Used*), útil para rastrear conexões ou processos ativos sem risco de esgotamento de memória (GREGG, 2020).

O acesso a mapas dentro de um programa eBPF é feito exclusivamente via *helper functions* como `bpf_map_lookup_elem`, `bpf_map_update_elem` e `bpf_map_delete_elem`, cujos parâmetros são verificados pelo *verifier* para garantir que os tamanhos de chave e valor respeitem os limites definidos na criação do mapa (Linux Kernel Documentation, 2024). Essa combinação de flexibilidade e verificação formal é o que permite construir sistemas de detecção e controle de segurança robustos sem abrir mão da estabilidade do kernel (GREGG, 2019).

5 MECANISMOS DE SEGURANÇA

O eBPF não é apenas uma ferramenta de observação passiva: integrado a subsistemas de segurança consolidados do kernel Linux, ele viabiliza o controle ativo de processos, redes e recursos do sistema com um nível de programabilidade que abordagens tradicionais não oferecem (Linux Kernel Documentation, 2024). Este capítulo apresenta os principais mecanismos pelos quais o eBPF interage com a infraestrutura de segurança do kernel.

5.1 EBPF LSM

O *Linux Security Module* (LSM) é uma interface do kernel que define um conjunto de *hooks* em pontos de decisão de segurança — abertura de arquivos, criação de processos, operações de rede, acesso a capacidades do sistema, entre outros — e permite que módulos de segurança interceptem e influenciem essas operações (Linux Kernel Documentation, 2024). Mecanismos tradicionais como SELinux e AppArmor são implementados como módulos LSM, porém requerem configuração complexa e não podem ser adaptados em tempo de execução.

A partir do kernel 5.7, o tipo de programa `BPF_PROG_TYPE_LSM` permite anexar programas eBPF diretamente aos *hooks* LSM, criando módulos de segurança totalmente programáveis (Linux Kernel Documentation, 2024). Ao retornar um valor de erro (por exemplo, `-EPERM`), o programa bloqueia a operação antes de sua conclusão; ao retornar zero, permite que ela prossiga. Essa decisão ocorre no próprio kernel, com latência de microssegundos, sem qualquer envolvimento do espaço de usuário no caminho crítico (GREGG, 2019).

Entre os *hooks* LSM mais relevantes para segurança estão:

- `bpf_lsm_file_open` — intercepta a abertura de arquivos, permitindo negar acesso com base em atributos do processo solicitante ou do arquivo;
- `bpf_lsm_bprm_check_security` — avalia a execução de um novo programa, viabilizando *allowlists* de binários autorizados;
- `bpf_lsm_socket_connect` — controla o estabelecimento de conexões de rede por processo, útil para isolar serviços comprometidos;
- `bpf_lsm_task_kill` — regula o envio de sinais entre processos, prevenindo interferências maliciosas entre contêineres.

Uma vantagem significativa do LSM BPF em relação ao SELinux e ao AppArmor é a possibilidade de combinar múltiplos programas no mesmo *hook* — graças ao mecanismo

BPF link — e de atualizar as políticas dinamicamente, sem reiniciar serviços ou carregar novos módulos de kernel (Linux Kernel Documentation, 2024).

5.2 *CGROUP HOOKS*

Control groups (cgroups) são o mecanismo do kernel Linux para organizar processos em hierarquias e aplicar limites de recursos (CPU, memória, I/O de rede e disco) a cada grupo (Linux Kernel Documentation, 2024). Em ambientes de contêineres, cada contêiner é tipicamente mapeado a um cgroup distinto, tornando os *hooks* de cgroup um ponto natural de aplicação de políticas de segurança com escopo por contêiner (GREGG, 2020).

O eBPF oferece um conjunto de tipos de programa associados a cgroups que permitem interceptar operações antes que elas aconteçam:

`BPF_PROG_TYPE_CGROUP_SKB`. Filtra pacotes na entrada ou saída de um cgroup específico. Permite implementar políticas de rede por contêiner sem depender de regras de *iptables* ou *nftables*, com menor overhead e maior expressividade (Linux Kernel Documentation, 2024).

`BPF_PROG_TYPE_CGROUP_SOCK` e variantes. Controla a criação e o uso de sockets por processos do cgroup. Pode bloquear a criação de sockets de determinados protocolos ou famílias de endereço, restringindo efetivamente a superfície de ataque de rede de um contêiner (GREGG, 2019).

`BPF_PROG_TYPE_CGROUP_DEVICE`. Regula o acesso a dispositivos (`/dev/*`) por processos do cgroup, substituindo com vantagens a lista de dispositivos do cgroups v1 (Linux Kernel Documentation, 2024).

`BPF_PROG_TYPE_CGROUP_SYSCTL`. Intercepta leituras e escritas em parâmetros `sysctl` realizadas por processos do cgroup, impedindo que um contêiner altere configurações globais do kernel que afetariam outros contêineres (Linux Kernel Documentation, 2024).

A granularidade por cgroup torna esses *hooks* particularmente adequados para ambientes Kubernetes, onde políticas de segurança precisam ser aplicadas individualmente a cada *pod* sem interromper os demais (CASSAGNES *et al.*, 2020).

5.3 SECCOMP E EBPF

O *secure computing mode* (seccomp) é um mecanismo do kernel Linux que restringe o conjunto de chamadas de sistema que um processo pode invocar (Linux Kernel

Documentation, 2024). No modo `SECCOMP_MODE_FILTER`, introduzido no kernel 3.5, um programa BPF clássico (cBPF) define a política de filtragem — especificando quais chamadas de sistema são permitidas, bloqueadas ou encaminhadas para análise em espaço de usuário (MCCANNE; JACOBSON, 1993).

Com o eBPF, dois aspectos dessa integração evoluíram significativamente:

Geração programática de filtros seccomp. Ferramentas modernas como `libseccomp` compilam políticas expressas em alto nível para o formato cBPF exigido pelo kernel. Pesquisas recentes exploram o uso de eBPF para gerar esses filtros de forma dinâmica a partir de perfis de comportamento observados em execução — em vez de listas estáticas definidas em tempo de desenvolvimento (GREGG, 2019).

Auditoria de violações seccomp com eBPF. Quando um filtro seccomp retorna `SECCOMP_RET_TRACE` ou `SECCOMP_RET_LOG`, a chamada de sistema bloqueada pode ser capturada por um programa eBPF conectado ao *tracepoint* `syscalls:sys_enter_*`, permitindo registrar tentativas de evasão com contexto completo — processo, PID, UID, argumentos — sem modificar o filtro seccomp original (Linux Kernel Documentation, 2024).

A combinação de seccomp com LSM BPF representa a abordagem de defesa em profundidade (*defense in depth*) recomendada para ambientes de produção: o seccomp restringe o conjunto de chamadas de sistema acessíveis, enquanto o LSM BPF impõe controles adicionais sobre o que pode ser feito com as chamadas permitidas (BEYER *et al.*, 2016). Juntos, os dois mecanismos reduzem drasticamente a superfície de ataque exposta por um processo comprometido (GREGG, 2020).

5.4 CONTROLE DE PROCESSOS E ACESSO A RECURSOS

Além dos mecanismos anteriores — orientados a interfaces específicas como sockets, arquivos e chamadas de sistema —, o eBPF oferece visibilidade e controle sobre o ciclo de vida de processos e o uso de recursos do sistema operacional de forma holística (GREGG, 2019).

Rastreamento de ancestralidade de processos. Utilizando mapas BPF do tipo `BPF_MAP_TYPE_HASH`, é possível manter em tempo real uma tabela de processos ativos, registrando PID, PPID, UID, executável e cgroup de cada processo (Linux Kernel Documentation, 2024). Essa árvore de processos viabiliza a detecção de *process injection* — em que um atacante insere código em um processo legítimo —, pois alterações inesperadas na identidade ou no comportamento de um processo tornam-se visíveis (MAJORS; FONG-JONES; MIRANDA, 2022).

Monitoramento de uso de capacidades (*capabilities*). O modelo de capacida-

des do Linux substitui o binômio root/não-root por um conjunto granular de privilégios (`CAP_NET_ADMIN`, `CAP_SYS_PTRACE`, `CAP_DAC_OVERRIDE`, entre outras) (Linux Kernel Documentation, 2024). Programas eBPF conectados ao *hook* LSM `bpf_lsm_capable` podem registrar toda tentativa de uso de capacidade, identificando processos que operam com mais privilégios do que o esperado — um indicador comum de escalonamento de privilégios (GREGG, 2020).

Controle de acesso a namespaces. Namespaces do Linux isolam recursos como PID, rede, sistema de arquivos e identificadores de usuário entre grupos de processos (Linux Kernel Documentation, 2024). Tentativas de um processo sair do seu namespace — por exemplo, via `setns(2)` ou `unshare(2)` — podem ser monitoradas e bloqueadas por programas eBPF, prevenindo fugas de contêineres (*container escapes*) (CASSAGNES *et al.*, 2020).

Observabilidade de acesso a arquivos e descritores. O *tracepoint* `sched_process_exit`, combinado com um mapa de estado de arquivos abertos, permite detectar processos que terminam sem fechar descritores sensíveis — padrão observado em algumas classes de *malware* que abusam de herança de descritores entre processos (GREGG, 2019).

Limitação dinâmica de recursos. Em conjunto com os *cgroup hooks* descritos na seção anterior, programas eBPF podem implementar cotas dinâmicas de chamadas de sistema por processo — por exemplo, limitando a taxa de chamadas `fork(2)` para mitigar ataques de exaustão de processos (*fork bombs*) — sem necessidade de reconfigurar os limites de cgroup em espaço de usuário (Linux Kernel Documentation, 2024).

A combinação desses controles com os mecanismos de observabilidade descritos no capítulo anterior cria uma camada de segurança coesa: eventos suspeitos são detectados, correlacionados e, quando necessário, bloqueados em uma única infraestrutura baseada em eBPF, sem a sobrecarga de agentes externos e com latência compatível com ambientes de produção (MAJORS; FONG-JONES; MIRANDA, 2022).

6 DETECÇÃO E MONITORAMENTO

A capacidade de detectar atividades maliciosas em tempo real, sem degradar o desempenho das aplicações monitoradas, é um dos principais diferenciais do eBPF frente a soluções tradicionais de segurança (CASSAGNES *et al.*, 2020). Este capítulo apresenta técnicas concretas de detecção e monitoramento baseadas em eBPF, organizadas pelos vetores de observação mais relevantes em ambientes de produção.

6.1 DETECÇÃO DE EXECUÇÃO DE PROCESSOS

A execução de processos é um dos eventos mais informativos para segurança: praticamente toda ação maliciosa — desde a exploração de uma vulnerabilidade até a persistência no sistema — envolve em algum momento a criação ou substituição de um processo (GREGG, 2019). O eBPF permite monitorar esse vetor com cobertura total e latência mínima, independentemente da origem do processo.

O *tracepoint* `sys_enter_execve` é acionado toda vez que um processo invoca a chamada de sistema `execve(2)`, responsável por substituir a imagem do processo atual por um novo executável (Linux Kernel Documentation, 2024). Um programa eBPF nesse ponto coleta, sem modificar o processo:

- PID e PPID — identificadores do processo e de seu pai, permitindo reconstruir a árvore de execução;
- UID e GID — identidade real do solicitante, revelando tentativas de execução sob identidade indevida;
- caminho do executável e argumentos da linha de comando, via `bpf_probe_read_user_str`;
- cgroup e namespace de PID, essenciais para identificar o contêiner de origem em ambientes Kubernetes (CASSAGNES *et al.*, 2020).

O exemplo a seguir ilustra um script bpftrace que registra toda execução originada de processos filhos de servidores web — padrão típico de exploração de vulnerabilidades de execução remota de código:

Listing 6.1 – Detecção de shell a partir de processo de serviço web com bpftrace.

```
tracepoint:syscalls:sys_enter_execve
/comm == "nginx" || comm == "apache2"/
{
    printf("[ALERTA] shell filho de %s: %s (pid=%d uid=%d)\n",
           comm, str(args->filename), pid, uid);
```

```
}
```

Além do `execve`, o *tracepoint* `sched_process_fork` captura a criação de processos via `fork(2)` e `clone(2)`, permitindo rastrear a propagação de processos maliciosos que se replicam antes de executar sua carga (GREGG, 2020). Combinados em um mapa BPF que mantém a árvore de parentesco em memória, esses dois pontos de observação fornecem uma visão completa do ciclo de vida de processos sem qualquer instrumentação das aplicações (MAJORS; FONG-JONES; MIRANDA, 2022).

6.2 MONITORAMENTO DE ARQUIVOS, CHAMADAS DE SISTEMA E CONEXÕES

Acesso a arquivos. Programas eBPF conectados ao *tracepoint* `sys_enter_openat` registram toda abertura de arquivo com contexto completo — processo, caminho, *flags* de acesso e resultado da operação (Linux Kernel Documentation, 2024). Caminhos de particular interesse para segurança incluem:

- `/etc/shadow`, `/etc/passwd` e `/root/.ssh/` — arquivos de credenciais frequentemente alvo de exfiltração;
- `/proc/$PID/mem` e `/proc/$PID/maps` — interfaces de memória de processo usadas por técnicas de injeção (Linux man-pages project, 2024);
- `/usr/bin/`, `/lib/` e `/sbin/` abertos em modo de escrita — indicativo de *rootkits* que modificam binários do sistema.

A ferramenta `opensnoop` do BCC (IO Visor Project, s.d.) implementa esse padrão e é amplamente utilizada em investigações forenses para identificar quais arquivos foram acessados durante um incidente.

Chamadas de sistema. O conjunto completo de *tracepoints* `sys_enter_*` oferece visibilidade sobre qualquer chamada de sistema invocada por qualquer processo (Linux Kernel Documentation, 2024). Para fins de detecção, chamadas críticas são monitoradas seletivamente para reduzir o volume de eventos:

- `ptrace` — interceptação de outro processo; abusada por *malware* para injeção de código e por *rootkits* para ocultar presença;
- `setuid` / `setgid` / `setcap` — alteração de identidade ou capacidades, indicativas de escalonamento de privilégios;
- `mmap` com `PROT_EXEC` — mapeamento de memória executável, padrão de carregamento de shellcode (GREGG, 2019);

- `init_module` / `finit_module` — carregamento de módulos de kernel, vetor clássico de instalação de *rootkits*.

Conexões de rede. Combinando *tracepoints* de *socket* com *hooks* de XDP e TC, o eBPF oferece visibilidade em múltiplas camadas da pilha de rede (Linux Kernel Documentation, 2024). O monitoramento de conexões de saída — via `sys_enter_connect` — permite identificar processos que estabelecem comunicação com endereços não previstos, padrão associado a *beaconing* de *Command and Control* (C2) e exfiltração de dados (CASSAGNES *et al.*, 2020). A ferramenta `tcpconnect` do BCC (IO Visor Project, s.d.) registra cada nova conexão TCP com PID, endereço de destino e porta, sem impacto perceptível sobre o throughput da rede (GREGG, 2019).

6.3 DETECÇÃO DE COMPORTAMENTO ANÔMALO

A detecção de anomalias vai além do monitoramento de eventos individuais: consiste em identificar sequências ou padrões de comportamento que diferem do que é esperado para um processo, serviço ou usuário (MAJORS; FONG-JONES; MIRANDA, 2022). O eBPF é particularmente adequado para esse fim porque permite acumular estado em mapas BPF ao longo do tempo e realizar correlações diretamente no kernel, antes mesmo que os eventos sejam enviados ao espaço de usuário (Linux Kernel Documentation, 2024).

Desvio de perfil de chamadas de sistema. Cada classe de aplicação — servidor web, banco de dados, agente de monitoramento — apresenta um perfil estável de chamadas de sistema em operação normal. Um programa eBPF pode construir, durante um período de observação inicial, um histograma de frequência de chamadas por processo e alertar quando a distribuição observada diverge significativamente desse perfil de referência (GREGG, 2019). Essa técnica é resistente a *malware* que tenta mimetizar nomes de processos legítimos, pois o comportamento em nível de chamadas de sistema é mais difícil de falsificar.

Comunicação lateral entre contêineres. Em ambientes Kubernetes, comunicação direta entre *pods* que normalmente não se comunicam é um indicador de movimentação lateral após comprometimento (CASSAGNES *et al.*, 2020). Mapas BPF do tipo `BPF_MAP_TYPE_HASH` indexados por par (cgroup de origem, cgroup de destino) permitem detectar esses fluxos anômalos em tempo real, sem depender de logs de *firewall* com atraso de segundos (Linux Kernel Documentation, 2024).

Escalada de privilégios em tempo de execução. A sequência `fork` → `execve` de um binário SUID → `setuid(0)` é o padrão de exploração de vulnerabilidades de escalonamento de privilégios local mais comum (GREGG, 2020). Um programa eBPF que rastreia transições de UID em mapas indexados por PID detecta essa sequência em

microssegundos, muito antes que ela se manifeste em logs de auditoria (GREGG, 2019).

Injeção de código em processos em execução. O uso de `ptrace` ou de `process_vm_writev` para escrever em memória de outro processo é uma técnica de injeção amplamente explorada (GREGG, 2019). Correlacionando o PID do processo chamador com o PID alvo e verificando se a relação pai-filho é legítima — usando a árvore de processos mantida em um mapa BPF — é possível distinguir operações legítimas de depuradores de tentativas de injeção maliciosa (MAJORS; FONG-JONES; MIRANDA, 2022).

6.4 AUDITORIA DE EVENTOS

A auditoria de segurança tem como objetivo registrar, de forma confiável e verificável, eventos relevantes para conformidade e investigação forense (BEYER *et al.*, 2016). O subsistema `auditd` do Linux é a solução tradicional para esse fim, mas apresenta limitações de desempenho em sistemas de alta carga e dificuldades de extensão para novos tipos de eventos (GREGG, 2020). O eBPF surge como complemento ou substituto natural: coleta eventos com menor sobrecarga, permite filtragem inteligente diretamente no kernel e é extensível sem alterações no sistema operacional (CASSAGNES *et al.*, 2020).

Registro de eventos com contexto completo. Por meio do mapa `BPF_MAP_TYPE_RINGBUF`, programas eBPF emitem registros de auditoria estruturados para o espaço de usuário com campos como timestamp em nanossegundos, PID, UID, cgroup, chamada de sistema, argumentos e código de retorno (Linux Kernel Documentation, 2024). Essa riqueza de contexto supera a granularidade oferecida pelo `auditd` para a maioria dos eventos (GREGG, 2019).

Filtragem inteligente no kernel. Diferentemente do `auditd`, que envia todos os eventos para o espaço de usuário e filtra depois, programas eBPF aplicam regras de interesse diretamente no kernel — registrando apenas eventos que satisfazem critérios de segurança predefinidos (Linux Kernel Documentation, 2024). Isso reduz drasticamente o volume de dados transmitidos e processados, tornando viável a auditoria contínua mesmo em sistemas com dezenas de milhares de chamadas de sistema por segundo (CASSAGNES *et al.*, 2020).

Integridade do registro de auditoria. Uma preocupação legítima em auditoria forense é a possibilidade de um atacante com privilégios suficientes suprimir registros de suas ações. Programas eBPF em execução no kernel são mais difíceis de desabilitar silenciosamente do que agentes em espaço de usuário: sua remoção requer privilégios explícitos e gera eventos observáveis por outros programas eBPF — criando uma cadeia de custódia parcialmente auto-referencial (MAJORS; FONG-JONES; MIRANDA, 2022).

Integração com sistemas externos. Registros de auditoria coletados por progra-

mas eBPF podem ser encaminhados para sistemas de gerenciamento de eventos e informações de segurança (SIEM) via exportadores como o `ebpf_exporter` (Cloudflare, s.d.) ou via *pipelines* Prometheus/Grafana (Prometheus Authors, s.d.; Grafana Labs, s.d.). Essa integração centraliza os dados de múltiplos nós em uma visão unificada, essencial para detecção de ataques distribuídos que envolvem vários sistemas simultaneamente (BEYER *et al.*, 2016).

7 FERRAMENTAS DE SEGURANÇA

O ecossistema de ferramentas construídas sobre eBPF cresceu rapidamente nos últimos anos, cobrindo desde utilitários interativos de linha de comando até plataformas de segurança em produção adotadas por grandes organizações (GREGG, 2019). Este capítulo apresenta as principais ferramentas relevantes para aplicações de segurança, organizadas por nível de abstração — da biblioteca de baixo nível à plataforma de detecção e resposta.

7.1 BCC

O *BPF Compiler Collection* (BCC) é um conjunto de ferramentas e uma framework de desenvolvimento para programas eBPF mantido pelo projeto IO Visor (IO Visor Project, s.d.). Ele foi pioneiro na democratização do eBPF ao introduzir uma API em Python e Lua sobre a qual programas eBPF em C são compilados e carregados em tempo de execução, eliminando a necessidade de configurar manualmente a cadeia de compilação clang/LLVM para cada ferramenta (GREGG, 2019).

O BCC inclui mais de cem ferramentas prontas para uso, das quais diversas têm aplicação direta em segurança:

- **`execsnoop`** — registra toda execução de processo com PID, PPID, UID e linha de comando, permitindo reconstruir cadeias de execução após um incidente;
- **`opensnoop`** — monitora abertura de arquivos em tempo real, essencial para detectar acesso a credenciais e arquivos de configuração sensíveis;
- **`tcpconnect`** e **`tcpaccept`** — registram conexões TCP de saída e entrada por processo, viabilizando a detecção de *beaconing* e de serviços não autorizados em escuta;
- **`capable`** — rastreia o uso de capacidades (*capabilities*) do Linux por processo, identificando tentativas de escalonamento de privilégios;
- **`ttysnoop`** — intercepta a saída de terminais TTY, útil para auditoria de sessões interativas de administradores ou atacantes.

Do ponto de vista arquitetural, cada ferramenta BCC é composta por dois componentes: o programa eBPF em C, compilado em tempo de execução e carregado no kernel, e um controlador em Python que configura os mapas BPF, recebe os eventos e os apresenta ao operador (IO Visor Project, s.d.). Essa separação facilita a criação de novas ferramentas, mas introduz a dependência do compilador LLVM em cada nó moni-

torado — limitação mitigada pelo uso de libbpf com BPF CO-RE, descrito na Seção 7.3 (NAKRYIKO, 2020).

7.2 BPFTRACE

O bpftrace é uma linguagem de rastreamento de alto nível para eBPF, inspirada em DTrace e AWK, que permite escrever programas de monitoramento e investigação em poucas linhas diretamente na linha de comando (bpftrace Contributors, s.d.). Enquanto o BCC é orientado ao desenvolvimento de ferramentas reutilizáveis, o bpftrace é otimizado para investigação interativa e *ad hoc* — ideal para resposta a incidentes e análise forense em tempo real (GREGG, 2019).

A linguagem oferece acesso direto a *tracepoints*, *kprobes*, *uprobes* e *perf events* por meio de uma sintaxe concisa de predicados e ações:

Listing 7.1 – Monitoramento de tentativas de abertura de `/etc/shadow` com bpftrace.

```
tracepoint:syscalls:sys_enter_openat
/str(args->filename) == "/etc/shadow"/
{
    printf("[ALERTA] %s (pid=%d uid=%d) tentou abrir /etc/shadow\n",
           comm, pid, uid);
}
```

Listing 7.2 – Detecção de uso de `ptrace` por processos não privilegiados.

```
tracepoint:syscalls:sys_enter_ptrace
/uid != 0/
{
    printf("[ALERTA] ptrace por %s (pid=%d uid=%d) -> pid alvo: %d\n",
           comm, pid, uid, args->pid);
}
```

O bpftrace suporta agregações nativas — histogramas, contagens e somas — que permitem sintetizar grandes volumes de eventos em estatísticas compactas sem sobrecarregar o espaço de usuário com eventos individuais (bpftrace Contributors, s.d.). Essa capacidade é valiosa em investigações de segurança onde o volume de eventos é alto e o operador precisa rapidamente identificar os processos ou caminhos mais frequentes em determinado vetor de acesso (GREGG, 2020).

7.3 LIBBPF

A libbpf é a biblioteca oficial de baixo nível para desenvolvimento de aplicações eBPF em C, mantida como parte do próprio repositório do kernel Linux e espelhada em repositório independente para facilitar o consumo por projetos externos (libbpf Contributors, s.d.). Ela abstrai as chamadas de sistema `bpf(2)` e `perf_event_open(2)`, oferecendo uma API estável para carregamento de programas, criação e acesso a mapas, e gerenciamento do ciclo de vida de *links* BPF (Linux Kernel Documentation, 2024).

O principal diferencial da libbpf em relação ao BCC é o suporte nativo ao BPF CO-RE (*Compile Once – Run Everywhere*) (NAKRYIKO, 2020). Com CO-RE, o programa eBPF é compilado uma única vez — gerando um binário portável — e ao ser carregado em diferentes versões do kernel, a libbpf reescreve automaticamente os offsets de acesso a estruturas do kernel com base nas informações de tipo BTF (*BPF Type Format*) expostas pelo sistema (Linux Kernel Documentation, 2024). Isso elimina a necessidade de distribuir o compilador LLVM junto com a ferramenta, viabilizando a implantação de agentes de segurança em frotas de servidores heterogêneas com kernels de versões distintas (NAKRYIKO, 2020).

O fluxo de desenvolvimento com libbpf segue o padrão:

1. Escrever o programa eBPF em C usando os *headers* `vmlinux.h` (gerado via `bpftool btf dump`) e `bpf/bpf_helpers.h`;
2. Compilar com `clang --target bpf -g -O2`, gerando um objeto `.o` contendo o *bytecode* BPF e metadados BTF;
3. Usar o *skeleton* gerado por `bpftool gen skeleton` para integrar o carregamento e a configuração do programa na aplicação em espaço de usuário em C (libbpf Contributors, s.d.).

Ferramentas de segurança de produção como Falco e Tetragon utilizam libbpf internamente, o que ilustra sua posição como base do ecossistema moderno de eBPF (Linux Kernel Documentation, 2024).

7.4 FALCO

O Falco é uma plataforma de segurança em tempo de execução (*runtime security*) de código aberto, originalmente desenvolvida pela Sysdig e hoje um projeto graduado da *Cloud Native Computing Foundation* (CNCF). Seu objetivo é detectar comportamentos anômalos e violações de política em contêineres, máquinas virtuais e hosts Linux por meio

de regras expressas em alto nível, traduzidas internamente para programas eBPF que monitoram chamadas de sistema (Linux Kernel Documentation, 2024; GREGG, 2019).

A arquitetura do Falco é composta por três camadas:

1. **Driver de coleta** — responsável por capturar eventos do kernel. O Falco suporta três modos: módulo de kernel (legado), driver eBPF clássico e, a partir da versão 0.35, driver moderno baseado em libbpf com suporte a CO-RE (NAKRYIKO, 2020; libbpf Contributors, s.d.);

2. **Motor de regras** — avalia cada evento capturado contra um conjunto de regras YAML que definem condições e campos de alerta. As regras padrão cobrem os vetores de ataque mais comuns em ambientes Kubernetes, como execução de shell em contêiner, acesso a segredos montados e modificação de binários do sistema;

3. **Canal de saída** — publica alertas em múltiplos destinos: *stdout*, arquivos de log, webhooks HTTP, filas de mensagens e integrações com sistemas SIEM (MAJORS; FONG-JONES; MIRANDA, 2022).

Um exemplo de regra Falco que detecta shell em contêiner é apresentado a seguir:

Listing 7.3 – Regra Falco para detecção de terminal interativo em contêiner.

```
- rule: Terminal shell in container
  desc: Detecta a abertura de shell interativo dentro de um conteiner.
  condition: >
    spawned_process and container
    and shell_procs and proc.tty != 0
  output: >
    Shell interativo detectado (usuario=%user.name
    container=%container.name imagem=%container.image.repository
    proc=%proc.cmdline)
  priority: WARNING
```

O Falco é amplamente adotado como primeira linha de detecção em clusters Kubernetes, onde a efêmeridade dos processos torna ineficazes as abordagens tradicionais de auditoria baseadas em logs (CASSAGNES *et al.*, 2020).

7.5 TETRAGON

O Tetragon é uma plataforma de segurança e observabilidade baseada em eBPF desenvolvida pela Isovalent — empresa criadora do projeto Cilium — e disponibilizada como código aberto. Diferentemente do Falco, que opera principalmente como detector passivo, o Tetragon integra observabilidade e **aplicação de políticas** (*enforcement*) em um

único agente: ele pode não apenas alertar sobre comportamentos suspeitos, mas bloqueá-los em tempo real diretamente no kernel, sem roundtrip para o espaço de usuário (Linux Kernel Documentation, 2024; GREGG, 2019).

O Tetragon expõe dois primitivos principais:

TracingPolicy. Um recurso Kubernetes que define, de forma declarativa, quais eventos coletar e quais ações tomar. Internamente, cada política é compilada em um ou mais programas eBPF que são carregados no kernel e conectados a *kprobes*, *tracepoints* ou *hooks* LSM conforme necessário. As ações disponíveis incluem `Sigkill` — enviar SIGKILL ao processo infrator diretamente do kernel — e `Override` — retornar um código de erro para a chamada de sistema sem permitir sua execução (Linux Kernel Documentation, 2024).

Exportador de eventos gRPC. Todos os eventos coletados são emitidos via gRPC em um formato estruturado rico em contexto — incluindo metadados de Kubernetes como namespace, *pod* e *label* — compatível com sistemas de análise como Hubble e plataformas SIEM (MAJORS; FONG-JONES; MIRANDA, 2022).

Uma *TracingPolicy* que bloqueia escrita em binários do sistema por qualquer processo, exceto o gerenciador de pacotes, ilustra a capacidade de *enforcement* do Tetragon:

Listing 7.4 – TracingPolicy Tetragon bloqueando escrita em diretórios de sistema.

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: proteger-binarios-sistema
spec:
  kprobes:
  - call: "security_file_permission"
    syscall: false
    args:
    - index: 0
      type: "file"
    - index: 1
      type: "int"
    selectors:
    - matchArgs:
      - index: 1
        operator: "Equal"
        values: ["2"]
      matchBinaries:
      - operator: "NotIn"
        values: ["/usr/bin/dpkg", "/usr/bin/rpm"]
      matchActions:
      - action: Sigkill
```

A combinação de Falco — para detecção com regras de alto nível — e Tetragon —

para aplicação de políticas de baixa latência — representa uma arquitetura de defesa em profundidade eficaz em ambientes de contêineres, onde a velocidade de resposta é crítica para conter ataques antes que se propaguem (BEYER *et al.*, 2016; CASSAGNES *et al.*, 2020).

8 EBPF EM CONTAINERS E KUBERNETES

A adoção massiva de contêineres e orquestradores como o Kubernetes trouxe novos desafios de segurança: ambientes altamente dinâmicos, com processos efêmeros, múltiplos inquilinos compartilhando o mesmo kernel e superfície de ataque distribuída por dezenas ou centenas de nós (CASSAGNES *et al.*, 2020). Abordagens tradicionais de segurança baseadas em agentes por processo ou em inspeção de logs com atraso mostram-se inadequadas nesse contexto. O eBPF, por operar no kernel compartilhado por todos os contêineres de um nó, oferece visibilidade e controle centralizados sem depender de instrumentação individual de cada carga de trabalho (Linux Kernel Documentation, 2024).

8.1 SEGURANÇA DE CONTÊINERES

Contêineres não são máquinas virtuais: compartilham o kernel do sistema hospedeiro e dependem de mecanismos de isolamento do próprio Linux — namespaces, cgroups e capacidades — para separar cargas de trabalho (Linux Kernel Documentation, 2024). Essa arquitetura torna o kernel o ponto central de controle e, consequentemente, o principal vetor de ataque em ambientes de contêineres (GREGG, 2020).

As ameaças mais relevantes nesse contexto são:

Fuga de contêiner (*container escape*). Um processo dentro de um contêiner que consegue sair do seu namespace de PID, de rede ou de sistema de arquivos e interagir com o kernel ou com outros contêineres sem restrição (Linux Kernel Documentation, 2024). Fugas exploram tipicamente vulnerabilidades em chamadas de sistema como `unshare(2)`, `setns(2)` ou `mount(2)`, ou abusam de capacidades Linux concedidas desnecessariamente ao contêiner.

Escalada de privilégios dentro do contêiner. Mesmo sem sair do namespace, um processo que obtém `CAP_SYS_ADMIN` ou executa como `root` no contêiner pode comprometer outros processos do mesmo nó (GREGG, 2019).

Ataques de movimentação lateral. Um contêiner comprometido que estabelece comunicação direta com outros contêineres ou serviços de infraestrutura — banco de dados, API do Kubernetes — fora dos fluxos previstos (CASSAGNES *et al.*, 2020).

O eBPF endereça esses vetores de múltiplas formas. Programas conectados a *tracepoints* de chamadas de sistema como `unshare`, `setns` e `clone` detectam tentativas de manipulação de namespaces em tempo real (Linux Kernel Documentation, 2024). *Hooks* LSM BPF podem bloquear essas operações para processos cujos cgroups não estejam em uma lista de permissões, sem qualquer modificação nas imagens de contêiner (GREGG, 2019). Mapas BPF indexados por par de cgroups registram todo fluxo de rede entre

contêineres, tornando a movimentação lateral imediatamente visível (MAJORS; FONG-JONES; MIRANDA, 2022).

8.2 SEGURANÇA EM KUBERNETES

O Kubernetes introduz uma camada adicional de complexidade: além dos processos em execução nos nós, existe o plano de controle — API Server, etcd, scheduler, controller manager — cujo comprometimento implica controle total sobre o cluster (BEYER *et al.*, 2016). O eBPF é utilizado em duas frentes complementares nesse ambiente.

Substituição do kube-proxy. O componente tradicional `kube-proxy` implementa as regras de roteamento de serviços Kubernetes por meio de tabelas *iptables* de crescimento linear com o número de serviços (Linux Kernel Documentation, 2024). Projetos como o Cilium substituem o `kube-proxy` por programas eBPF anexados a *hooks* TC e XDP, implementando o mesmo roteamento com complexidade constante e menor latência, além de oferecer visibilidade sobre cada fluxo de rede entre *pods* (CASSAGNES *et al.*, 2020).

Observabilidade do plano de dados. Em um cluster com centenas de *pods* espalhados por dezenas de nós, rastrear um fluxo de requisição de ponta a ponta requer correlação de eventos de múltiplos processos e nós (MAJORS; FONG-JONES; MIRANDA, 2022). Programas eBPF nos nós coletam identificadores de fluxo — endereços IP, portas, metadados de cgroup — e os associam a metadados Kubernetes (namespace, nome do *pod*, *labels*) obtidos do API Server, compondo uma visão de observabilidade de rede enriquecida sem necessidade de *sidecars* ou modificação das aplicações (Linux Kernel Documentation, 2024).

Auditoria do API Server. Toda operação no cluster — criação de *pods*, modificação de segredos, concessão de permissões RBAC — passa pelo API Server e pode ser auditada por programas eBPF conectados aos *tracepoints* de rede do nó onde ele executa, complementando os logs de auditoria nativos do Kubernetes com contexto de processo e identidade de rede (GREGG, 2019).

8.3 POLÍTICAS DE SEGURANÇA

O Kubernetes oferece primitivos nativos de política de segurança — *NetworkPolicies*, *PodSecurityAdmission* e RBAC — mas sua implementação e granularidade dependem dos plugins de infraestrutura instalados (Linux Kernel Documentation, 2024). O eBPF amplia significativamente o espaço de políticas aplicáveis.

Políticas de rede por identidade. Enquanto *NetworkPolicies* nativas operam so-

bre endereços IP e portas, que mudam a cada recriação de *pod*, soluções baseadas em eBPF como o Cilium implementam políticas baseadas em identidade: a identidade de um *pod* é derivada dos seus *labels* Kubernetes e codificada em um rótulo numérico propagado em cabeçalhos de pacotes (CASSAGNES *et al.*, 2020). Isso permite expressar regras como "*pods* do serviço A só podem receber tráfego de *pods* do serviço B no mesmo namespace" de forma estável, independentemente de endereços IP efêmeros (Linux Kernel Documentation, 2024).

Políticas de chamadas de sistema por *pod*. Perfis seccomp restringem as chamadas de sistema disponíveis para um contêiner, mas são estáticos e difíceis de calibrar sem causar falsos positivos (GREGG, 2020). O eBPF permite gerar perfis dinâmicos observando o comportamento real de cada *pod* durante um período de aprendizado e, em seguida, aplicar uma política derivada desse perfil sem modificar o manifesto do *pod* (NAKRYIKO, 2020).

***TracingPolicies* declarativas.** Ferramentas como o Tetragon expõem um recurso Kubernetes (`TracingPolicy`) que permite expressar políticas de segurança em YAML — quais chamadas de sistema monitorar, quais argumentos filtrar e qual ação tomar (alertar, registrar ou bloquear) — que são traduzidas automaticamente em programas eBPF e implantadas em todos os nós do cluster (Linux Kernel Documentation, 2024). Esse modelo declarativo integra a segurança ao fluxo de *GitOps* já adotado por equipes de plataforma, tratando políticas de segurança como código versionável e revisável (BEYER *et al.*, 2016).

8.4 DETECÇÃO DE ATIVIDADES SUSPEITAS

A natureza dinâmica do Kubernetes — onde *pods* são criados e destruídos em segundos — exige que a detecção de atividades suspeitas opere em tempo real e seja capaz de correlacionar eventos efêmeros antes que evidências desapareçam (MAJORS; FONG-JONES; MIRANDA, 2022). O eBPF é a tecnologia que melhor satisfaz esse requisito, e os padrões de ataque mais comuns em ambientes Kubernetes têm contrapartidas diretas em programas de detecção baseados em eBPF.

Execução de shell em contêiner. A abertura de um terminal interativo (`/bin/sh`, `/bin/bash`) dentro de um contêiner de produção é um forte indicador de comprometimento ou de acesso não autorizado via `kubectl exec` (CASSAGNES *et al.*, 2020). Programas eBPF detectam esse padrão combinando o *tracepoint* `sys_enter_execve` com a verificação de que o processo possui um TTY associado e que seu cgroup corresponde a um *pod* em execução (Linux Kernel Documentation, 2024).

Acesso a segredos montados. Segredos Kubernetes são montados como arquivos no sistema de arquivos do contêiner. Leitura desses arquivos por processos diferentes do

esperado — especialmente por shells ou binários de sistema — é detectada por programas eBPF conectados ao *tracepoint* `sys_enter_openat`, filtrando caminhos que correspondam a diretórios de montagem de segredos (`/var/run/secrets/`) (GREGG, 2019).

Comunicação com o API Server a partir de contêineres de aplicação. Requisições HTTP ao endereço do API Server (`kubernetes.default.svc`) originadas de *pods* de aplicação que não deveriam ter acesso ao plano de controle indicam possível tentativa de movimentação para comprometer o cluster (BEYER *et al.*, 2016). Mapas BPF indexados por endereço de destino e cgroup de origem permitem detectar esses fluxos em microssegundos e bloqueá-los via *hook* de cgroup de socket (Linux Kernel Documentation, 2024).

Modificação de imagens em tempo de execução. Contêineres devem ser imutáveis: qualquer escrita no sistema de arquivos raiz fora de volumes montados explicitamente é suspeita (GREGG, 2020). Programas eBPF conectados a *hooks* LSM de abertura de arquivo em modo de escrita — filtrados pelo namespace de montagem do contêiner — detectam esse padrão independentemente da ferramenta ou técnica usada pelo atacante para realizar a escrita (Linux Kernel Documentation, 2024).

Varredura de rede interna. Um *pod* comprometido que tenta conectar a múltiplos endereços e portas em sequência rápida executa varredura de rede para mapear serviços internos do cluster (CASSAGNES *et al.*, 2020). Mapas BPF do tipo `BPF_MAP_TYPE_LRU_HASH` acumulam contagens de tentativas de conexão por *pod* em janelas de tempo deslizantes e acionam alertas quando o número de destinos distintos excede um limiar configurável (Linux Kernel Documentation, 2024; GREGG, 2020).

9 EXEMPLOS PRÁTICOS

Este capítulo reúne três cenários de investigação que combinam as ferramentas apresentadas no Capítulo 7 — BCC e bpftrace — com as técnicas de detecção do Capítulo 6. Cada cenário parte de uma pergunta de segurança, mostra os comandos para respondê-la e indica como interpretar o resultado. Todos os comandos exigem `root` e foram pensados para uma máquina de laboratório, nunca para um sistema em produção sem autorização. As ferramentas prontas são as do pacote `libbpf-tools`, que no Ubuntu têm o sufixo `-libbpf`. As versões do BCC (`-bpfcc`) falharam ao compilar no kernel 7.0 usado nos testes, com `static assertion failed` em `include/linux/fs.h`, e por isso não são usadas aqui.

9.1 MONITORAMENTO DE PROCESSOS E ARQUIVOS

Pergunta: que processos estão sendo executados e quem está tentando ler arquivos de credenciais?

O ponto de partida é o `execsnoop`, que registra cada `execve` com PID, PPID e linha de comando. Em um terminal, deixe-o em execução; em outro, rode comandos quaisquer e veja-os aparecer.

Listing 9.1 – Execuções de processos com o `execsnoop-libbpf`.

```
sudo execsnoop-libbpf
```

Para os arquivos, o `opensnoop` lista cada abertura. Como a saída é volumosa, filtre pelos caminhos sensíveis:

Listing 9.2 – Aberturas de arquivos sensíveis com o `opensnoop`.

```
sudo opensnoop-libbpf | grep --line-buffered -E '/etc/(shadow|passwd)'
```

Se o objetivo é alertar apenas sobre um arquivo específico, um programa bpftrace filtra no próprio kernel, reduzindo o volume de eventos. É o mesmo programa do Capítulo 7; salve-o como `shadow.bt`:

Listing 9.3 – `shadow.bt`: alerta de abertura de `/etc/shadow`.

```
tracepoint:syscalls:sys_enter_openat
/str(args->filename) == "/etc/shadow"/
{
    printf("[ALERTA] %s (pid=%d uid=%d) tentou abrir /etc/shadow\n",
           comm, pid, uid);
}
```

Listing 9.4 – Execução do alerta e geração de um evento de teste.

```
sudo bpftrace shadow.bt
# em outro terminal:
head -c1 /etc/shadow
```

Executado por um usuário comum, o `head -c1 /etc/shadow` falha com "Permissão negada", mas o alerta aparece mesmo assim: o *tracepoint* `sys_enter_openat` dispara na entrada da chamada de sistema, antes de o kernel decidir se o acesso é permitido. Isso é útil, pois tentativas malsucedidas também indicam reconhecimento.

Nos testes, a captura foi irregular. Com quatro programas abrindo `/etc/shadow` como usuário comum, o alerta disparou para o `head` e para o `python3`, mas não para o `cat` (no Ubuntu 26.04, a versão do projeto uutils) nem para um redirecionamento do `bash`, embora o `strace` mostre que os três chamam o mesmo `openat(AT_FDCWD, "/etc/shadow", O_RDONLY...)`. O `opensnoop` também não registrou nenhuma das aberturas na linha filtrada. A causa não foi investigada. A consequência prática é que um alerta baseado em um único *tracepoint* pode perder eventos, e a ausência de alerta não prova que o arquivo não foi acessado.

O ponto de atenção está nos falsos positivos: processos legítimos, como `sshd` e `login`, leem esses arquivos o tempo todo. Em um uso real, acrescente ao predicado uma lista do que é esperado (`comm`) e alerte apenas sobre o resto.

9.2 RASTREAMENTO DE CHAMADAS DE SISTEMA

Pergunta: que chamadas de sistema um processo suspeito está usando, e alguém está usando chamadas associadas a escalonamento de privilégios ou injeção?

Primeiro, uma visão geral: contar as chamadas por processo mostra quem é o mais ativo e dá uma linha de base para a detecção de anomalias do Capítulo 6. O mapa é impresso ao encerrar com `Ctrl-C`.

Listing 9.5 – Contagem de chamadas de sistema por processo.

```
sudo bpftrace -e 'tracepoint:raw_syscalls:sys_enter { @[comm] = count();
   ↪  }'
```

Em seguida, monitore chamadas específicas. O programa abaixo, também do Capítulo 7, alerta sobre `ptrace` por usuários não privilegiados. Salve-o como `ptrace.bt`:

Listing 9.6 – `ptrace.bt`: uso de `ptrace` por usuários não root.

```
tracepoint:syscalls:sys_enter_ptrace
/uid != 0/
{
    printf("[ALERTA] ptrace por %s (pid=%d uid=%d) -> pid alvo: %d\n",
```

```
            comm, pid, uid, args->pid);
}
```

Para gerar um evento de teste, use o `strace`, que se apoia em `ptrace`:

Listing 9.7 – Execução do alerta e geração de um evento de teste.

```
sudo bpftrace ptrace.bt
# em outro terminal, como usuario comum:
strace -f true
```

A carga de módulos de kernel é outra chamada que merece alerta permanente, por ser o caminho clássico de instalação de *rootkits* (Capítulo 6). Em condições normais ela é rara, então qualquer disparo merece atenção:

Listing 9.8 – Alerta de carga de módulos de kernel.

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_finit_module {
    printf("[ALERTA] %s (pid=%d uid=%d) carregou um modulo\n", comm, pid
    ↪ , uid);
}'
```

Nos testes, esse alerta não disparou quando o módulo `dummy` foi carregado com `modprobe`, nem mesmo ao vigiar também o `init_module`, a outra chamada de sistema usada para carregar módulos. A causa não foi investigada, e o resultado se soma à captura irregular observada na seção anterior: um alerta baseado em um único *tracepoint* pode perder eventos.

Como em qualquer filtro baseado em nomes de processo, `comm` é controlado pelo próprio processo e pode ser falsificado. Para decisões de bloqueio, prefira critérios que o atacante não controle, como o UID ou o cgroup.

9.3 DETECÇÃO DE CONEXÕES SUSPEITAS

Pergunta: que processos estão abrindo conexões de saída, e para onde?

O `tcpconnect` registra cada nova conexão TCP de saída com PID, processo, endereço e porta de destino:

Listing 9.9 – Conexões TCP de saída com o `tcpconnect`.

```
sudo tcpconnect-libbpf
```

Em outro terminal, gere uma conexão para observar o registro. O exemplo usa o recurso `/dev/tcp` do Bash para abrir uma conexão local sem depender de outras ferramentas:

Listing 9.10 – Geração de uma conexão de teste.

```
bash -c 'exec 3<>/dev/tcp/127.0.0.1/22'
```

Para uma visão que não depende do BCC, o *tracepoint* `sys_enter_connect` também registra as tentativas, ainda que sem o endereço decodificado:

Listing 9.11 – Tentativas de conexão com bpftrace.

```
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_connect {
    printf("%s (pid=%d uid=%d) chamou connect()\n", comm, pid, uid);
}'
```

Para interpretar a saída, procure conexões fora do padrão: processos que nunca deveriam usar a rede, como um servidor de banco de dados abrindo conexões de saída, e repetições em intervalos regulares para o mesmo destino, característica de *beaconing* (Capítulo 6). O `tcpconnect` não diz se a conexão é maliciosa; ele fornece a evidência para a investigação.

10 SEGURANÇA DO PRÓPRIO EBPF

O eBPF é frequentemente apresentado como uma tecnologia inerentemente segura por construção, e há razões sólidas para isso: o verificador estático, o modelo de capacidades e o sandboxing no kernel constituem um conjunto de garantias superior ao de módulos de kernel tradicionais (Linux Kernel Documentation, 2024). No entanto, como qualquer tecnologia que opera com privilégios elevados no coração do sistema operacional, o eBPF introduz sua própria superfície de ataque — e compreendê-la é essencial para quem pretende utilizá-lo em ambientes de produção ou de segurança (GREGG, 2019).

10.1 PRIVILÉGIOS NECESSÁRIOS

Historicamente, carregar qualquer programa eBPF exigia a capacidade `CAP_SYS_ADMIN`, o privilégio mais amplo do modelo de capacidades Linux — equivalente, na prática, a acesso de superusuário (Linux Kernel Documentation, 2024). Essa exigência limitava o uso do eBPF a administradores de sistema e tornava inviável sua adoção em ambientes multiusuário onde processos não privilegiados precisassem de observabilidade.

A partir do kernel 5.8, o modelo foi refinado com a introdução de capacidades mais granulares (Linux Kernel Documentation, 2024):

- **`CAP_BPF`** — permite carregar programas eBPF e criar mapas BPF de propósito geral, sem conceder os demais privilégios associados a `CAP_SYS_ADMIN`;
- **`CAP_PERFMON`** — necessária para anexar programas a eventos de desempenho (`perf events`), *kprobes* e *tracepoints*, em conjunto com `CAP_BPF`;
- **`CAP_NET_ADMIN`** — exigida para programas que operam sobre a pilha de rede, como os do tipo XDP e TC.

Além das capacidades, o parâmetro do kernel `kernel.unprivileged_bpf_disabled` controla se processos sem nenhuma das capacidades acima podem carregar programas eBPF do tipo `SOCKET_FILTER` e `CGROUP_SKB` — os únicos historicamente acessíveis a processos não privilegiados (Linux Kernel Documentation, 2024). Em ambientes de segurança, recomenda-se definir esse parâmetro como `1` para desabilitar o acesso não privilegiado ao eBPF:

Listing 10.1 – Desabilitação de eBPF não privilegiado via sysctl.

```
sudo sysctl -w kernel.unprivileged_bpf_disabled=1
```

Em ambientes de contêineres, a concessão de `CAP_BPF` a um contêiner deve ser tratada com cautela: um contêiner com essa capacidade pode, em princípio, instrumentar processos de outros contêineres no mesmo nó, comprometendo o isolamento esperado (GREGG, 2020).

10.2 EBPF VERIFIER

O verificador (*verifier*) é a principal linha de defesa do kernel contra programas eBPF maliciosos ou defeituosos (Linux Kernel Documentation, 2024). Conforme descrito no Capítulo 4, ele realiza análise estática sobre o grafo de fluxo de controle do programa antes de qualquer execução. Neste capítulo, o foco recai sobre suas limitações e sobre os casos em que ele pode ser contornado ou enganado.

Complexidade e superfície de ataque do próprio verificador. O verificador é um componente de software com dezenas de milhares de linhas de código e lógica altamente complexa de análise de tipos e intervalos de valores (Linux Kernel Documentation, 2024). Vulnerabilidades no verificador já foram descobertas e exploradas: um programa eBPF cuidadosamente construído pode enganar a análise de intervalos (*range analysis*), levando o verificador a aprovar código que, em execução, realiza acesso fora dos limites de um mapa ou registrador (GREGG, 2019). Essas vulnerabilidades são particularmente graves porque, uma vez aprovado pelo verificador, o programa passa a executar com plena confiança do kernel.

Vulnerabilidades históricas notáveis. Diversas CVEs relacionadas ao verificador eBPF foram divulgadas nos últimos anos, permitindo escalonamento de privilégios locais a partir de um processo com `CAP_BPF` ou, em kernels mais antigos, a partir de processos completamente não privilegiados (Linux Kernel Documentation, 2024; GREGG, 2020). Esses incidentes reforçam a importância de manter o kernel atualizado e de restringir o acesso ao subsistema BPF ao menor conjunto possível de processos.

Limites da verificação formal. O verificador não garante correção semântica — apenas propriedades estruturais de segurança e terminação (Linux Kernel Documentation, 2024). Um programa eBPF pode ser aprovado pelo verificador e ainda assim produzir resultados incorretos, registrar eventos falsos ou introduzir condições de corrida em mapas BPF compartilhados entre múltiplas CPUs sem o uso adequado de operações atômicas (GREGG, 2019). Essas falhas não comprometem o kernel, mas podem comprometer a integridade dos dados de segurança coletados.

10.3 RISCOS DE SEGURANÇA

A adoção do eBPF como infraestrutura de segurança introduz um paradoxo: a mesma tecnologia usada para defender sistemas pode ser weaponizada por atacantes com privilégios suficientes (GREGG, 2020).

eBPF como vetor de *rootkit*. Um atacante que obtém `CAP_BPF` ou `CAP_SYS_ADMIN` pode carregar programas eBPF que interceptam chamadas de sistema para ocultar arquivos, processos ou conexões de rede — implementando um *rootkit* inteiramente em eBPF, sem modificar o sistema de arquivos ou carregar módulos de kernel (GREGG, 2019). Diferentemente de *rootkits* tradicionais, que deixam rastros em módulos carregados ou em binários modificados, um *rootkit* eBPF reside apenas em memória e é mais difícil de detectar com ferramentas forenses convencionais (MAJORS; FONG-JONES; MIRANDA, 2022).

Exfiltração de dados via mapas BPF. Programas eBPF têm acesso de leitura à memória de processos via `bpf_probe_read`, incluindo buffers de aplicações que processam dados sensíveis — chaves criptográficas, senhas, tokens de autenticação (GREGG, 2019). Um programa malicioso pode copiar esses dados para um mapa BPF e exfiltrá-los para o espaço de usuário de forma praticamente invisível para ferramentas de monitoramento que não inspecionam o próprio subsistema BPF (Linux Kernel Documentation, 2024).

Interferência com programas eBPF legítimos. Em sistemas onde múltiplos programas eBPF operam simultaneamente — agentes de segurança, ferramentas de observabilidade, plugins de rede —, um programa malicioso com os privilégios adequados pode modificar mapas BPF compartilhados, alterar o comportamento de programas legítimos ou desanexar *links* BPF que implementam políticas de controle de acesso (Linux Kernel Documentation, 2024).

Ataques especulativos (*Spectre*). O compilador JIT do kernel gera código nativo a partir do *bytecode* BPF. Programas eBPF podem ser construídos para explorar vulnerabilidades de execução especulativa como Spectre, lendo memória do kernel fora dos limites permitidos pelo verificador via canais laterais de temporização (Linux Kernel Documentation, 2024). O kernel implementa mitigações como o mascaramento de índices de mapa e a serialização de acessos especulativos, mas essas mitigações impõem sobrecarga de desempenho e podem não cobrir todas as variantes de ataques especulativos (GREGG, 2020).

10.4 LIMITAÇÕES

Além dos riscos ativos de segurança, o eBPF apresenta limitações técnicas e operacionais relevantes que devem ser consideradas ao planejar seu uso em infraestrutura de segurança.

Dependência de versão de kernel. Embora o BPF CO-RE (NAKRYIKO, 2020) mitigue significativamente problemas de portabilidade, funcionalidades mais recentes — como LSM BPF, *ringbuf* e suporte a iteradores — só estão disponíveis em kernels recentes (Linux Kernel Documentation, 2024). Organizações que operam com distribuições empresariais de ciclo de vida longo, como RHEL 8 ou Ubuntu 20.04, podem não ter acesso a esses recursos sem atualizar o kernel, o que em ambientes regulados pode ser um processo demorado.

Tamanho e complexidade dos programas. O verificador impõe um limite no número de instruções por programa eBPF (historicamente 4.096, expandido para 1 milhão a partir do kernel 5.2 para programas privilegiados) e exige que todos os caminhos de execução sejam verificáveis (Linux Kernel Documentation, 2024). Lógica de detecção complexa pode não caber em um único programa e precisa ser distribuída entre múltiplos programas encadeados via *tail calls* ou *BPF-to-BPF calls*, aumentando a complexidade de desenvolvimento e depuração (GREGG, 2019).

Ausência de sistema de arquivos persistente. Programas e mapas eBPF residem na memória do kernel e são destruídos quando o último descritor de arquivo que os referencia é fechado (Linux Kernel Documentation, 2024). Embora o sistema de arquivos virtual `/sys/fs/bpf` permita fixar (*pin*) objetos BPF para persistência além do ciclo de vida do processo criador, a gestão desse estado persistente adiciona complexidade operacional a agentes de segurança que precisam sobreviver a reinicializações de serviços (GREGG, 2020).

Depuração limitada. O ambiente de execução eBPF não suporta chamadas de sistema arbitrárias, nem a maioria das funções de biblioteca padrão, o que torna a depuração de programas eBPF mais trabalhosa do que a de aplicações convencionais (GREGG, 2019). Ferramentas como `bpftool` e a integração com `llvm-objdump` auxiliam na inspeção do *bytecode* e do estado de mapas, mas o ciclo de desenvolvimento é significativamente mais lento do que em espaço de usuário (IO Visor Project, s.d.).

Cobertura limitada em ambientes altamente virtualizados. Em máquinas virtuais aninhadas ou em contêineres com configurações de kernel restritivas, funcionalidades do eBPF podem estar indisponíveis ou parcialmente desabilitadas (Linux Kernel Documentation, 2024). Organizações que operam em nuvens públicas devem verificar quais tipos de programa eBPF são suportados pelo hipervisor e pela versão de kernel disponível na instância, pois nem todas as ofertas de nuvem expõem o conjunto completo de

funcionalidades do eBPF ao sistema convidado (GREGG, 2020).

Curva de aprendizado. O desenvolvimento eficaz de programas eBPF exige compreensão simultânea de internos do kernel Linux, do modelo de programação BPF, das restrições do verificador e das ferramentas de compilação (GREGG, 2019). Equipes de segurança sem experiência prévia em desenvolvimento de kernel tendem a depender de ferramentas de alto nível como Falco e Tetragon, o que é adequado para a maioria dos casos de uso, mas limita a capacidade de customização quando as regras disponíveis não cobrem ameaças específicas do ambiente (MAJORS; FONG-JONES; MIRANDA, 2022).

11 CONCLUSÃO

Este manual percorreu o eBPF sob a ótica da segurança. Partimos dos fundamentos (Capítulos 1 a 4), passamos pelos mecanismos que o kernel oferece para controle ativo, como LSM, *cgroup hooks* e seccomp (Capítulo 5), e pelas técnicas de detecção e auditoria (Capítulo 6). Em seguida, vimos as ferramentas que materializam essas técnicas (Capítulo 7) e como elas se estendem a contêineres e Kubernetes (Capítulo 8).

O Capítulo 9 reuniu essas peças em três cenários de investigação, e o Capítulo 10 lembrou que o eBPF também é superfície de ataque: os mesmos privilégios que o tornam poderoso exigem `CAP_BPF` restrito, o *verifier* ativo e atenção às limitações de cada ferramenta. Detectar e bloquear com eBPF só é confiável quando o próprio mecanismo é protegido.

Os comandos do Capítulo 9 estão reunidos no script `testes/08-seguranca-exemplos.sh`, na pasta `testes/` do repositório, para serem executados em uma máquina de laboratório com `sudo`. Quem for adiante pode combinar o que aprendeu aqui com o Manual 1, de observabilidade, para enviar os eventos de segurança a um painel, ou ler a documentação do Falco e do Tetragon para levar essas regras a um ambiente real.

REFERÊNCIAS

ALVES, R. C.; PLINTA, J. P. S. H. **Monitoramento de Funções Virtualizadas de Redes: Um Estudo Comparativo entre Ferramentas de Extração de Comportamento**. Pinhais: Instituto Federal do Paraná, 2026. Trabalho de Conclusão de Curso (Bacharelado em Ciência da Computação).

BEYER, B. *et al.* **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol: O'Reilly Media, 2016.

bpftrace Contributors. **bpftrace: High-level tracing language for Linux**. s.d. Disponível em: <https://github.com/bpftrace/bpftrace>. Acesso em: 6 out. 2026.

CASSAGNES, C. *et al.* The rise of eBPF for non-intrusive performance monitoring. In: **NOMS 2020 – 2020 IEEE/IFIP Network Operations and Management Symposium**. [S.l.]: IEEE Press, 2020. p. 1–7. Disponível em: <https://doi.org/10.1109/NOMS47738.2020.9110434>.

Cloudflare. **ebpf_exporter: Prometheus exporter for custom eBPF metrics and OpenTelemetry traces**. s.d. Disponível em: <https://github.com/cloudflare/ebpf_exporter>. Acesso em: 6 out. 2026.

Grafana Labs. **Grafana documentation**. s.d. Disponível em: <https://grafana.com/docs/grafana/latest/>. Acesso em: 6 out. 2026.

GREGG, B. **BPF Performance Tools**. Boston: Addison-Wesley, 2019.

_____. **Systems Performance: Enterprise and the Cloud**. 2. ed. Boston: Addison-Wesley, 2020.

IO Visor Project. **BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more**. s.d. Disponível em: <https://github.com/iovisor/bcc>. Acesso em: 6 out. 2026.

libbpf Contributors. **libbpf: Automated upstream mirror for libbpf stand-alone build**. s.d. Disponível em: <https://github.com/libbpf/libbpf>. Acesso em: 6 out. 2026.

Linux Kernel Documentation. **BPF Documentation**. 2024. Disponível em: <https://www.kernel.org/doc/html/latest/bpf/>.

Linux man-pages project. **proc(5) — Linux manual page**. 2024. Disponível em: <https://man7.org/linux/man-pages/man5/proc.5.html>.

MAJORS, C.; FONG-JONES, L.; MIRANDA, G. **Observability Engineering: Achieving Production Excellence**. Sebastopol: O'Reilly Media, 2022.

MCCANNE, S.; JACOBSON, V. The BSD packet filter: A new architecture for user-level packet capture. In: **Proceedings of the USENIX Winter 1993 Conference**. San Diego: [s.n.], 1993. p. 259–270.

NAKRYIKO, A. **BPF CO-RE (Compile Once – Run Everywhere)**. 2020. Disponível em: <https://nakryiko.com/posts/bpf-portability-and-co-re/>.

PLINTA, J. P.; NAHAS, B. R. **ebpf-scitec: minicurso introdutório de eBPF**. 2026. Repositório de código. Disponível em: <https://github.com/joaopedroplinta/ebpf-scitec>.

Prometheus Authors. **Prometheus: Overview**. s.d. Disponível em: <https://prometheus.io/docs/introduction/overview/>. Acesso em: 6 out. 2026.